

PROJECT PROPOSAL & BILL OF MATERIALS

AURA

Autonomous Utility Rover Architecture — a modular, AI-powered mobile robotics platform for intelligent indoor navigation and future robotic manipulation.

Version 1.0 · Future Roadmap to Version 3.0

TEAM

3 Members

VERSION 1.0

July – October

VERSION 2.0

November onwards

EST. TOTAL COST

₹3.40–3.60 lakh

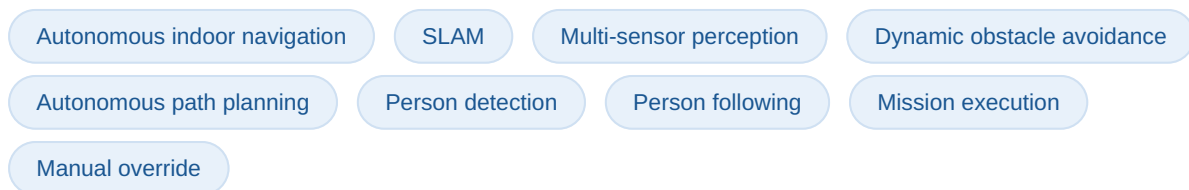
01 Project Overview & Vision

AURA (Autonomous Utility Rover Architecture) is a modular AI-powered autonomous mobile robotics platform designed for intelligent indoor navigation and future robotic manipulation.

Unlike traditional autonomous cars that perform only one task, AURA is designed as a complete robotics platform capable of mapping, localization, obstacle avoidance, person detection, mission execution and future teleoperation. The platform is designed from the ground up to be modular, so that additional systems such as robotic arms, teleoperation, exoskeleton control and AI assistants can be added without redesigning the entire robot.

Project Objective

To develop a fully autonomous six-wheel mobile robotics platform capable of:



Problem Statement

Current educational autonomous vehicle projects generally focus on a single capability such as lane following, parking or obstacle avoidance. The objective of AURA is to build a complete robotics platform capable of understanding unknown indoor environments, creating maps, localizing itself, planning safe paths and autonomously executing missions — while remaining modular for future research in manipulation and human-robot interaction.

Final Project Goal

AURA is not intended to be just another autonomous vehicle. It is designed as a long-term modular robotics platform that begins as an intelligent autonomous rover and gradually evolves into a fully capable mobile manipulator with teleoperation, AI-assisted autonomy and learning capabilities. The modular engineering approach allows future upgrades without redesigning the mobility platform, making it suitable for continued development throughout the remainder of the undergraduate program and beyond.

02 Deliverables by Version

Version 1.0 — October

Autonomous Mobility

- Six-wheel autonomous rover
- Manual mode
- Assisted mode
- Fully autonomous mode

Navigation

- Indoor SLAM & localization
- Path planning & waypoint navigation
- Mission execution
- Return Home

Artificial Intelligence

- YOLO object detection
- Person detection
- Follow registered user

Sensor Head

- Intel RealSense D455
- Two Degree-of-Freedom Pan-Tilt Head

Version 2.0 — November onwards

Two SO-101 Robotic Arms

DIY Exoskeleton Controller

Head Tracking

DJI Teleoperation

Voice Interaction

Local Large Language Model

Learning from Demonstration

Intelligent Manipulation

03 System Architecture

Mission commands flow from the human operator through the Jetson compute layer — which handles all perception, navigation and planning — down to the ESP32 microcontroller and motor controllers that drive the 6WD rover.



04 Compute, Sensing & Hardware

Main Computer — NVIDIA Jetson Orin Nano

ROS2 · AI · Navigation · SLAM · Object Detection · Path Planning · Mission Planning · Camera Processing

Microcontroller — ESP32

Motor control · Servo control · Sensor reading · Communication with Jetson · Emergency control

Intel RealSense D455

RGB & depth camera for object and person detection; future arm vision and face recognition. Mounted on a 2-DOF pan-tilt head.

2D LiDAR

Indoor mapping · localization · navigation · obstacle detection.

IMU

Orientation · angular velocity · robot stability.

Wheel Encoders

Odometry · speed measurement · distance measurement.

Drive System

Six-wheel drive rover powered by six planetary BLDC motors with encoders and dedicated BLDC motor controllers, running on 150 mm rubber wheels.

Power System

36 V battery pack with a Battery Management System, DC-DC buck converters, a power distribution board and an emergency stop switch.

Chassis — Custom Modular Aluminium

Features a modular electronics bay, battery compartment, sensor head mount and internal cable routing, with future arm mounts, a Pixhawk mount and expansion ports built in.

05 Software Stack & Sensor Responsibilities

Operating System Ubuntu 22.04	Middleware ROS2
Navigation Nav2 · RTAB-Map	Artificial Intelligence YOLO · OpenCV
Programming Languages Python · C++	Sensor Fusion Inputs LiDAR · RealSense · IMU · Wheel Encoders

SENSOR	PRIMARY RESPONSIBILITIES
LiDAR	Mapping, localization, obstacle detection
RealSense D455	RGB vision, depth vision, object detection
IMU	Robot orientation
Wheel Encoders	Odometry

06 Robot Modes & Navigation Workflow

Manual Mode

Robot controlled using keyboard or controller.

Assisted Mode

Operator drives the robot; the robot automatically avoids collisions.

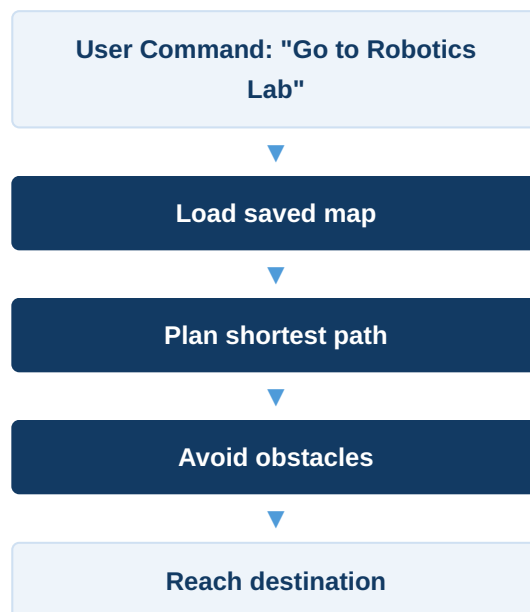
Autonomous Mode

Robot performs complete missions independently.

First Time in a New Environment

The robot detects an unknown environment and begins autonomous exploration. LiDAR builds a map while the RealSense supplies depth information, and the IMU and encoders refine localization. The generated map is saved, and the user labels important locations — for example: *Classroom 401*, *Classroom 402*, *Robotics Lab*, *Charging Station*, *Main Entrance*. These names are permanently stored.

Future Navigation



Mission Examples

1 · Patrol Laboratory

2 · Follow User

3 · Navigate to Charging Station

4 · Inspect Room

5 · Return Home

07 Development Timeline

PHASE 1	Project Foundation Requirements · Architecture · Git repository · BOM · CAD planning · Wiring diagram
PHASE 2	Mechanical Design Chassis · Suspension · Electronics mounts · Head · Battery housing
PHASE 3	Electronics Motors · Power · ESP32 · Jetson · Sensors
PHASE 4	Software ROS2 · SLAM · Navigation · AI · Sensor fusion
PHASE 5	Testing Indoor mapping · Navigation · Obstacle avoidance · Person following · Mission execution

08 Bill of Materials

Version 1.0 Components

COMPONENT	QTY	EST. COST (₹)
Custom Aluminium Chassis	1	12,000
Planetary BLDC Motors with Encoders	6	36,000
BLDC Motor Controllers	6	18,000
150 mm Rubber Wheels	6	6,000
Intel RealSense D455	1	55,000
2D LiDAR	1	25,000
9-Axis IMU	1	1,500
ESP32	1	800
36V Battery Pack	1	18,000
Battery Charger	1	5,000

COMPONENT	QTY	EST. COST (₹)
DC-DC Converters	2	2,000
Power Distribution Board	1	2,500
Emergency Stop	1	500
Wiring and Connectors	—	4,000
Bearings, Fasteners and Hardware	—	5,000
Cooling Fans	—	1,500
Version 1.0 Total (excl. Jetson Orin Nano)	—	₹1.90–2.00 lakh

Version 2.0 Additional Components

COMPONENT	QTY	EST. COST (₹)
Waveshare SO-101 Robotic Arm	2	56,000
Pan-Tilt Head Assembly	1	5,000
High Torque Head Servos	2	4,000
Microphone Array	1	3,000
Speaker	1	2,000
DIY Exoskeleton Electronics	1	20,000
IMUs for Exoskeleton	10–16	15,000
Rotary Encoders	12–16	12,000
Exoskeleton Microcontrollers	2	2,000
Additional Battery Capacity	1	10,000
Additional Power Electronics	—	5,000
Structural Hardware	—	10,000
Miscellaneous and Contingency	—	15,000
Version 2.0 Additional Total	—	₹1.50–1.60 lakh

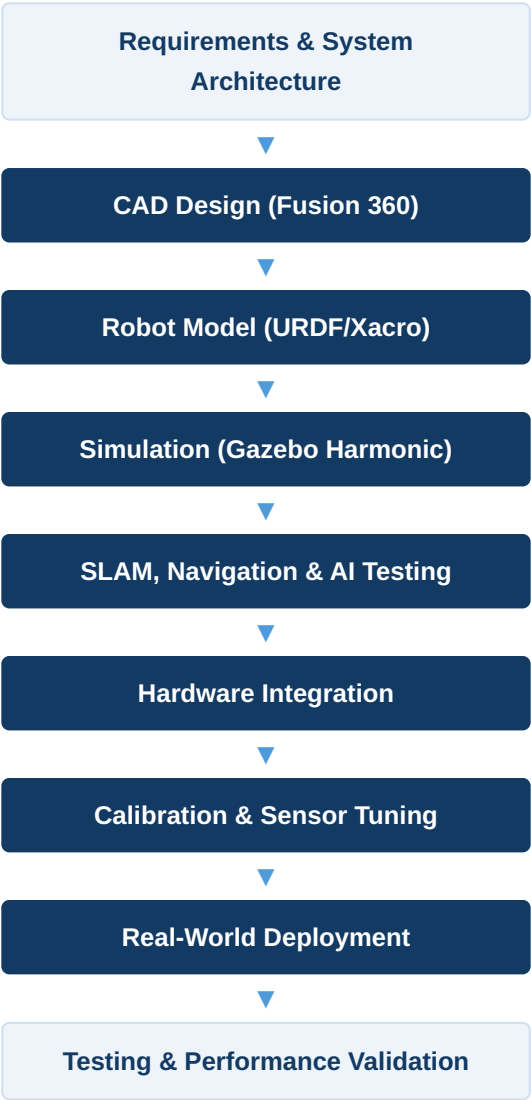
Total Project Cost

VERSION 1.0	VERSION 2.0 UPGRADE	COMPLETE ROBOT
₹1.90–2.00 L	₹1.50–1.60 L	₹3.40–3.60 L

Items already available (not included in cost): NVIDIA Jetson Orin Nano · DJI Goggles 3 · DJI O4 Pro Air Unit · 3D Printer · Printing Materials.

09 Development Workflow (Simulation-to-Real)

AURA will follow an industry-standard Simulation-to-Real (Sim-to-Real) development workflow. Instead of developing directly on hardware, every major subsystem will first be designed, tested and validated in simulation before deployment to the physical robot. This approach minimizes hardware damage, reduces debugging time and enables parallel software and mechanical development.



Phase 0 — Digital Twin

Before assembling the physical robot, a complete virtual representation (Digital Twin) of AURA will be created. This allows software development to begin immediately, even before all hardware components have arrived. This phase includes finalizing the system architecture; designing the chassis and mechanical components in Fusion 360; creating the robot description using URDF/Xacro; simulating sensors including LiDAR, Intel RealSense, IMU and wheel encoders; testing SLAM, localization, navigation and obstacle avoidance in simulation; and verifying software before hardware assembly.

10 Software Development Workflow

The project will follow a modular development approach where each subsystem is developed independently before full integration.



11 AI & Development Toolchain

The project will leverage modern AI-assisted software development while relying on established robotics frameworks instead of training core algorithms from scratch.

ChatGPT

- Robotics architecture
- System design
- Electronics planning
- ROS 2 integration
- Navigation logic
- Debugging & documentation

Claude

- Large-scale Python development
- ROS 2 package implementation
- C++ development
- Code refactoring
- Documentation
- Repository organization

GitHub Copilot

- Real-time code completion
- Boilerplate generation
- Function implementation
- Unit testing assistance

Robotics Software Stack

- ROS 2 · Gazebo Harmonic
- Nav2 · RTAB-Map
- OpenCV · YOLO
- Python · C++
- Git & GitHub

Future Software Stack

As the project evolves, additional technologies may be integrated. These components are planned for later stages and are **not required for Version 1.0**.

NVIDIA Isaac ROS

NVIDIA Isaac Sim

TensorRT optimization

MoveIt 2

Hugging Face LeRobot

Offline Voice AI

Local Large Language Models

12 Machine Learning Strategy

The project will prioritize proven, pre-trained models and established robotics software to accelerate development. The following components will be used without training from scratch:

Used As-Is (Version 1.0)

- Nav2 for autonomous navigation
- RTAB-Map for SLAM
- Pre-trained YOLO models for object detection
- OpenCV for computer vision

Future Work

- Custom object detection models
- Learning from Demonstration (LeRobot)
- Teleoperation-assisted data collection
- Imitation learning
- Autonomous manipulation

This approach reduces development time while ensuring a reliable and scalable robotics platform.

13 Team Roles & Responsibilities

Member 1 — Mechanical & CAD

- Chassis, sensor mounts, battery compartment & electronics enclosure in Fusion 360
- Pan-tilt camera mechanism
- Future robotic-arm mounting points
- Manufacturing drawings & assembly

Deliverables: Complete CAD assembly · Manufacturing drawings · Mechanical BOM

Member 2 — Electronics & Embedded

- Electrical architecture & complete wiring
- Motor/controller selection · ESP32 integration
- Power distribution & battery management
- Sensor wiring · emergency stop

Deliverables: Wiring diagram · Electrical BOM · Power system · Embedded firmware

Member 3 — AI, Navigation & Software

- Configure Ubuntu and ROS 2
- Develop SLAM · configure Nav2
- Integrate LiDAR & Intel RealSense
- Integrate YOLO
- Develop Mission Manager
- Test autonomous navigation

Deliverables: ROS 2 packages · Navigation stack · AI stack · Git repository · Documentation

14 Version 1 Scope (October)

The following features must be completed before October.

Mechanical

- Complete chassis
- 6WD drive system
- Pan-tilt camera head
- Battery mounting
- Electronics enclosure

Electronics

- Jetson Orin Nano · ESP32
- LiDAR · Intel RealSense · IMU
- Wheel encoders
- Power distribution
- Emergency stop

Software

- ROS 2
- SLAM
- Nav2
- Localization
- Path planning
- Obstacle avoidance
- Mission Manager
- Person detection
- Person following
- Return Home

Demonstration

The robot should be capable of creating a map of an unknown environment, saving the generated map, loading a previously created map, navigating autonomously, avoiding static and moving obstacles, detecting people, following a registered user and returning to the charging position.

15 Version 2 Scope (Future)

These features are intentionally postponed until after Version 1 is stable. **Version 1 will not be delayed to implement these features.**

Dual SO-101 robotic arms

DIY exoskeleton

DJI teleoperation

Head tracking

Voice interaction

Offline AI assistant

Learning from Demonstration (LeRobot)

Autonomous manipulation

16 Project Milestones

MILESTONE

1

Project Planning Completed

Requirements finalized · architecture finalized · BOM finalized.

MILESTONE

2

Mechanical Design Completed

CAD completed · chassis finalized · wiring planned.

MILESTONE

3

Simulation Completed

Robot model created · SLAM tested · navigation tested · mission execution tested.

MILESTONE

4

Hardware Integration Completed

Electronics assembled · sensors operational · motors operational.

MILESTONE

5

Autonomous Navigation Completed

Mapping · localization · obstacle avoidance · person following.

MILESTONE

6

Final Demonstration

Robot successfully completes all Version 1 objectives.

17 Repository Structure & Coding Standards

```
AURA/
├── README.md
├── LICENSE
├── .gitignore
├── docs/
│   ├── Proposal/      ├── Meeting Notes/  └── Reports/
│   ├── Research/      └── Wiring Diagrams/
├── cad/
│   ├── Chassis/  ├── Head/  ├── Mounts/  ├── Arms/ (Future)  └── Drawings/
├── electronics/
│   ├── Schematics/  ├── PCB/  ├── Power/  └── BOM/
├── firmware/
│   └── esp32/
├── software/
│   ├── ros2_ws/  ├── navigation/  ├── slam/  ├── perception/
│   ├── mission_manager/  ├── teleop/ (Future)  └── utils/
├── simulation/
│   ├── urdf/  ├── meshes/  ├── worlds/  ├── gazebo/  └── isaac_sim/ (Future)
├── ai/
│   ├── yolo/  ├── datasets/  ├── models/  └── training/ (Future)
├── testing/
├── media/
│   ├── images/  ├── videos/  └── renders/
└── resources/
```

Coding Standards

Use Git for version control and commit frequently with meaningful messages. Every new feature should have its own branch, and broken code must never be committed to the main branch. Document all major changes, and keep all wiring diagrams, CAD files and documentation inside the repository.

18 Success Criteria

Version 1 will be considered complete when the robot can:

- ✓ Power on reliably.
- ✓ Build a map of an unknown indoor environment.

- ✓ Save and reload maps.
- ✓ Navigate autonomously between user-defined locations.
- ✓ Detect and avoid static obstacles.
- ✓ Detect and avoid moving obstacles.
- ✓ Detect and follow a registered user.
- ✓ Return to its home location.
- ✓ Switch between Manual, Assisted and Autonomous modes.

19 Phase 1 — Foundation

Goal: before buying parts or writing code, create a professional project structure and lock the core decisions.

Step 1 — Git Repository

Initialize the **AURA** repository using the professional folder structure defined in Section 17 (Repository Structure), separating documentation, CAD, electronics, firmware, software, simulation, AI, testing, media and resources — with reserved (*Future*) folders for arms, teleoperation, Isaac Sim and training.

Step 2 — Software Stack (Frozen)

Operating System Ubuntu 22.04	Robotics ROS 2 Humble
Navigation Nav2 · RTAB-Map	Vision OpenCV · Ultralytics YOLO
Simulation Gazebo Harmonic	CAD Fusion 360

Step 3 — Version 1 Features (Frozen)

These are the only features built before October; everything else is postponed.

Mobility

- 6WD Rover
- Manual · Assisted · Autonomous modes

Navigation

- SLAM · Mapping · Localization
- Path Planning · Return Home

Vision

- Person Detection
- Object Detection
- Person Following

Hardware

- Pan-Tilt Head · RealSense
- LiDAR · IMU · Encoders

Step 4 — Version 2 (Frozen / Postponed)

Do not work on these until Version 1 is complete.

SO-101 Arms

Exoskeleton

DJI Teleoperation

Voice Assistant

Local LLM

Learning from Demonstration

Step 5 — Mechanical Design: Questions Before CAD

Answer these before opening Fusion 360 — only then start CAD:

1. Final dimensions
2. Wheel size
3. Motor selection
4. Battery location
5. Sensor locations
6. Jetson location
7. ESP32 location
8. LiDAR location
9. Future arm mounting
10. Cable routing

Step 6 — Phase 1 Deliverables

- ✓ Final system architecture
- ✓ Locked BOM
- ✓ Folder structure
- ✓ Git repository
- ✓ Initial CAD layout
- ✓ Wiring diagram
- ✓ Power budget
- ✓ Software architecture
- ✓ Development roadmap

20 Ways of Working & Open-Source Platform

Rather than jumping around, AURA will be built exactly like a robotics company — progressing through clear engineering milestones and treating the project as an open-source robotics platform, not just a college project.

Engineering Milestones

- 1 · System Architecture (*current*)
- 2 · Mechanical Design
- 3 · Electronics
- 4 · Simulation
- 5 · Embedded Software

- 6 · ROS 2
- 7 · AI & Navigation
- 8 · Hardware Integration
- 9 · Testing & Validation
- 10 · Version 1 Complete

Maintained as an Open-Source Platform

Professional Git repository

Versioned BOM

CAD revisions

Engineering change log

Wiring documentation

Software documentation

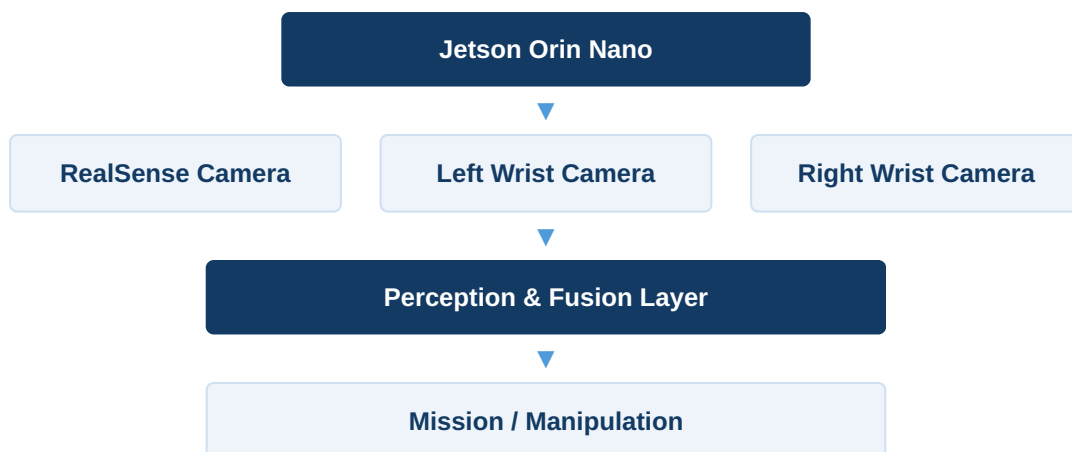
Assembly guide

Calibration guide

User manual

21 Multi-Camera Perception Architecture

For fluid manipulation, cameras must run **simultaneously** — never one replacing another. Switching cameras, even for a second, would make the manipulation experience feel terrible. The architecture is therefore designed so all cameras stream at once and the system simply changes which camera is *prioritized* for a given task.



What Happens During a Pick

1. RealSense detects the bottle from across the room.
2. Robot navigates to the table.
3. As the arm approaches, the wrist camera is already streaming.
4. The manipulation software starts using the wrist camera for fine alignment.
5. The RealSense doesn't stop — it continues monitoring the overall scene for people, obstacles and changes.

There is no "camera switch." It is a change in which camera is given priority, with every input available at the same time:

SOURCE	CONCURRENT SIGNAL
RealSense	"The bottle is on the table."
Wrist camera	"The gripper is 15 mm too far left."
IMU + encoders	"The robot is stable."
Joint encoders	"The arm is at the correct pose."

Can the Jetson Orin Nano Handle This?

Yes — provided the workload is managed sensibly. Heavy AI inference does not need to run on every camera at full resolution all the time:

- **RealSense** — navigation and global perception (always on).
- **Left wrist camera** — processed only when the left arm is manipulating.
- **Right wrist camera** — processed only when the right arm is manipulating.

Recommendation — A Perception Manager

Rather than cameras feeding directly into robot logic, route them through a dedicated Perception Manager:



The Perception Manager decides which camera(s) to use for navigation, which to use for grasping, and when to fuse information from multiple cameras.

The "One Organism" Behavior

The robot never "switches eyes." It uses whichever eyes are most useful for the current task while keeping the others available in the background — behaving as a single coordinated organism rather than a set of competing sensors.

22 Version 1 Freeze & Weekly Execution Plan

The Only Question That Matters (Now → October)

Everything is judged by one question: "Does this help us complete Version 1?" If the answer is no, it is parked for Version 2.

Version 1 Definition (Frozen)

An autonomous 6-wheel indoor rover capable of building a map, localizing itself, navigating autonomously, avoiding obstacles and following a person. Everything else is an extension.

Version 1 Hardware

Included

- 6WD chassis
- 6 motors + encoders
- Jetson Orin Nano · ESP32
- LiDAR · Intel RealSense D455 · IMU
- Battery & power system
- Pan-tilt camera head

Not Yet

No arms

No exoskeleton

No wrist cameras

No DJI integration

The chassis is still designed with mounting points and cable routes so these can be added later **without redesigning the robot.**

Version 1 Software

ROS 2

Gazebo simulation

URDF/Xacro

Nav2

RTAB-Map

YOLO

Mission Manager

Dashboard (basic)

Version 1 Dashboard

Only the essentials — no arm controls yet:

Monitoring

- Live RealSense feed
- SLAM map & robot position
- Battery level
- CPU/GPU usage
- Sensor health · System logs

Control

- Robot mode (Manual / Assisted / Autonomous)
- Start mission
- Return Home
- Emergency Stop

Simplified October Architecture

To keep the codebase focused, the October architecture is deliberately simplified around a single Mission Manager:

```
Mission Manager
|
|— Navigation Manager
|— Perception Manager
|— Hardware Manager
|— Dashboard Manager
|— Safety Manager
```

Deliberately absent — not deleted, simply not implemented yet:

Manipulation Manager

Teleoperation Manager

This Week

DAY

1

Finalize hardware · create Git repository · create project structure.

DAY

2

Design chassis layout · decide exact dimensions · place electronics.

DAY

3

Build the CAD model · start the URDF.

DAY

4

Spawn the robot in Gazebo · verify kinematics.

DAY

5

Set up ROS 2 · test simulated movement.

Immediate Goal — End of Week

A complete CAD concept, a ROS 2 workspace, a robot model in Gazebo, a basic local dashboard, and a digital AURA driving in simulation. Once that foundation works, moving to the physical rover becomes far more straightforward because the software and hardware are already aligned.

23 Long-Term Roadmap

Version 1

- Autonomous mobility
- Indoor SLAM
- Autonomous navigation
- Multi-sensor fusion
- Mission execution
- Person following

Version 2

- Dual SO-101 robotic arms
- DIY exoskeleton teleoperation
- Head tracking
- VR telepresence
- Offline AI assistant
- Learning from Demonstration (LeRobot)

Version 3

- Autonomous mobile manipulation
- Multi-step task planning
- Advanced human-robot interaction
- Multi-room semantic mapping
- Intelligent task execution
- Fully modular robotics research platform

24 Long-Term Vision & Project Philosophy

AURA is designed as a modular robotics platform rather than a single-semester project. Future upgrades will include robotic manipulation, immersive teleoperation, wearable exoskeleton control, offline conversational AI and Learning from Demonstration — all while preserving the same mobility platform and software architecture. The objective is to continue developing AURA throughout the remainder of the undergraduate program into a research-grade autonomous mobile manipulation platform.

Build once. Upgrade forever.

Every hardware and software decision should prioritize modularity, maintainability and future expansion. No subsystem should require redesign simply to add new capabilities.

The robot should evolve through plug-and-play modules while preserving a stable, reliable autonomous core.